Taller 2: análisis de ADN con hebras en Python

Guía paso a paso de los ejercicios 2, 3, 4 y 5 · Versión para acompañar un repositorio de GitHub

Qué contiene el proyecto

Archivo	Utilidad
taller2_estudio.py	Código ejecutable: versión secuencial, cuatro hebras, verificación con assert y mediciones.
Taller2_guia_codigo_GitHub.pdf	Explicación de las tareas, las decisiones y los errores típicos.
README.md	Instrucciones para ejecutar y publicar el material como repositorio de estudio.

Mapa de los ejercicios

Apartado	Qué debes aprender
2 · Secuencial	Reunir cuatro análisis en un diccionario.
3 · Concurrente	Crear cuatro hebras, proteger escrituras y esperar resultados.
4 · Rendimiento	Medir medianas, speedup y eficiencia.
5 · Defensa	Justificar las decisiones y los límites de la estrategia.

Nota de uso académico. La pauta original prohíbe utilizar IA generativa para resolver o redactar la entrega evaluada. Este proyecto se presenta explícitamente como material de estudio asistido por IA: úsalo para practicar y construir tu propia explicación, no como entrega independiente.

Fuente de referencia: Taller 2 (Unidad 2), Universidad de Talca; código de inicio taller.py. Las funciones iniciales fueron conservadas sin cambios.

02 / Implementación secuencial

El programa recibe una lista de secuencias de ADN y ejecuta cuatro funciones ya definidas en taller.py. En esta versión los cálculos se hacen uno después de otro.

Operación	Qué cuenta
analizar_gc	Secuencias cuyo contenido G+C es al menos 55 %.
analizar_atg	Todas las apariciones de ATG; una secuencia puede aportar varias.
analizar_tata	Secuencias que contienen TATA al menos una vez.
analizar_longitud	Cantidad total de nucleótidos entre las secuencias.

Función que reúne los cuatro cálculos

```
def analizar_secuencial(secuencias):
    return {
        "gc_alto": analizar_gc(secuencias),
        "atg": analizar_atg(secuencias),
        "tata": analizar_tata(secuencias),
        "longitud_total": analizar_longitud(secuencias),
    }
```

Ejemplo para comprobar a mano

```
datos = ["ATGC", "GGCC", "TATA", "ATGATG"]
print(analizar_secuencial(datos))
```

Resultado: `{'gc_alto': 1, 'atg': 3, 'tata': 1, 'longitud_total': 18}`

Explicación: GGCC es la única secuencia con GC ≥ 55 %. ATG aparece una vez en ATGC y dos en ATGATG; TATA aparece en una secuencia; la longitud total es 4 + 4 + 4 + 6 = 18.

03 / Implementación concurrente

La estrategia elegida es descomposición funcional: cuatro hebras realizan cuatro cálculos independientes sobre la misma lista. Las hebras comparten memoria dentro del proceso.

Trabajador	Tarea
Hebra-gc_alto	analizar_gc
Hebra-atg	analizar_atg
Hebra-tata	analizar_tata
Hebra-longitud_total	analizar_longitud

Preparar las tareas y los trabajadores

```
import threading

def analizar_hebras(secuencias, mostrar_hebras=False):
    resultados = {}
    errores = []
    candado = threading.Lock()

    # Aquí se define la función interna trabajar(...).
    # Está completa en el archivo taller2_estudio.py.

    tareas = [
        ("gc_alto", analizar_gc),
        ("atg", analizar_atg),
        ("tata", analizar_tata),
        ("longitud_total", analizar_longitud),
    ]
```

Cómo trabaja cada hebra

```
def trabajar(clave, funcion):
    try:
        valor = funcion(secuencias)  # Sin bloquear el cálculo
    except Exception as exc:
        with candado:
            errores.append((clave, repr(exc)))
        return
    with candado:
        resultados[clave] = valor
```

Por qué Lock: protege las escrituras a las estructuras compartidas. La lectura de las secuencias y el cálculo de cada función ocurren fuera del candado, para no serializar todo el trabajo. La implementación incluye avisos opcionales y manejo de errores de hebras en el archivo .py.

03 / Iniciar, esperar y comprobar

La diferencia crítica entre concurrencia posible y ejecución secuencial está en el orden de start() y join(). Se deben iniciar todas las hebras antes de esperar a ninguna.

```
hebras = []
for clave, funcion in tareas:
    h = threading.Thread(
        target=trabajar,
        args=(clave, funcion),
        name=f"Hebra-{clave}"
    )
    hebras.append(h)

for h in hebras:
    h.start()
for h in hebras:
    h.join()

if errores:
    raise RuntimeError(f"Un trabajador falló: {errores}")
return resultados
```

Instrucción	Significado
Thread(target=..., args=...)	Construye una hebra, pero no inicia la función.
start()	Inicia esa hebra.
join()	Hace esperar al principal hasta que termine.
resultados[clave] = valor	Guarda un resultado; la clave distingue a cada operación.

Comparación correcta de los resultados

```
datos = generar_secuencias(20000, 400)
esperado = analizar_secuencial(datos)
obtenido = analizar_hebras(datos)
print("Secuencial:", esperado)
print("Hebras:", obtenido)
assert obtenido == esperado
```

Atención: el resultado del diccionario puede mostrarse en distinto orden: el orden de llegada de las hebras no cambia la igualdad de los diccionarios. Si aparece NameError, revisa el nombre de la función; si aparece AssertionError, imprime ambos diccionarios para ver qué clave o valor difiere.

04 / Medición e interpretación

La pauta exige mínimo tres mediciones por implementación y utilizar la mediana. La generación de las secuencias debe ocurrir antes de iniciar el cronómetro.

```
import statistics
import time

def medir(funcion, datos, repeticiones=3):
    tiempos = []
    for _ in range(repeticiones):
        inicio = time.perf_counter()
        funcion(datos)
        tiempos.append(time.perf_counter() - inicio)
    return statistics.median(tiempos)

ts = medir(analizar_secuencial, datos)
tp = medir(analizar_hebras, datos)
speedup = ts / tp
eficiencia = speedup / 4
```

Fórmulas solicitadas

Indicador	Fórmula / interpretación
Tiempo secuencial (Ts)	Mediana de los tiempos de analizar_secuencial.
Tiempo concurrente (Tp)	Mediana de los tiempos de analizar_hebras.
Trabajadores (p)	4 hebras en esta propuesta.
Speedup (S)	S = Ts / Tp. Si es mayor que 1, la versión concurrente fue más rápida.
Eficiencia (E)	E = S / p. Multiplicar por 100 para mostrar el porcentaje.

Interpretar sin inventar resultados

La computadora y la configuración de Python afectan las mediciones. La compilación free-threaded permite que las hebras ejecuten código Python en paralelo cuando el GIL está desactivado, pero eso no garantiza aceleración. Los costos de coordinación y la carga desigual pueden hacer que la versión concurrente tarde más.

Por eso el PDF no presenta valores de rendimiento como si hubieran sido medidos en el equipo del estudiante. El archivo .py calcula e imprime las medianas reales cada vez que se ejecuta.

05 / Defensa de las decisiones

Si el laboratorio procesara solo 100 secuencias

Conviene considerar la versión secuencial: crear cuatro hebras, iniciarlas y sincronizarlas tiene un costo que podría superar el tiempo del análisis. La elección definitiva requiere medir.

Si aumentara a un millón de secuencias

Se podría evaluar la descomposición por datos, dividiendo el conjunto en bloques y asignando grupos a los trabajadores; luego habría que combinar los resultados parciales. También importan memoria disponible, núcleos lógicos y costo de coordinación.

Preguntas típicas del profesor

Pregunta	Idea fundamental
¿Qué puede solaparse?	Las cuatro operaciones; ninguna depende del resultado de las otras.
¿Cómo vuelven los resultados?	Las hebras escriben en un mismo diccionario utilizando claves distintas.
¿Por qué utilizar Lock?	Para controlar la escritura a estructuras compartidas y proteger la consistencia.
¿Qué ocurriría sin join?	El programa principal podría leer resultados incompletos.
¿Por qué 4 hebras no implican 4 veces más velocidad?	Tareas desiguales, recursos disponibles y costo de sincronización.

Distinción importante

Concurrencia significa que distintas tareas progresan durante un intervalo. Paralelismo significa que ejecutan al mismo tiempo. Una tarea independiente puede implementarse secuencialmente o concurrentemente; ser independiente no la vuelve paralela por sí sola.

Ejecutar y preparar el repositorio

Comandos de PowerShell

```
python taller2_estudio.py
python taller2_estudio.py --cantidad 100 --mostrar-hebras
py -V:3.15t taller2_estudio.py
```

La versión por defecto crea 20.000 secuencias de longitud 400 con semilla fija 2026. El programa imprime resultados secuenciales y concurrentes, verifica su igualdad y muestra mediana, speedup y eficiencia.

Comprobación rápida del GIL

```
py -V:3.15t -c "import sys; print(sys._is_gil_enabled())"
```

Deberías ver False si el GIL está desactivado en ese intérprete. En caso de que el comando simple python seleccione otra instalación, utiliza py -V:3.15t.

Resultados verificables del ejemplo base

```
gc_alto:         487
atg:          124284
tata:          15461
longitud_total: 8000000
```

Son resultados deterministas de las funciones originales para 20.000 secuencias de 400 bases y semilla 2026. No constituyen una medición de rendimiento.

Problemas frecuentes

Síntoma	Revisión
NameError al llamar analizar_hebras	Confirma que su def aparece antes de main y que el nombre es idéntico.
AssertionError al comparar	Imprime ambos diccionarios; confirma cuatro claves y GC ≥ 55 %.
No aparece nada en pantalla	Assert no imprime cuando es verdadero; utiliza print.
GIL True	Comprueba el intérprete free-threaded o ejecútalo explícitamente.

GitHub: crea un repositorio de estudio, incorpora el archivo .py, este PDF y README.md. Señala claramente la asistencia de IA y los recursos utilizados.